



Take-Home Assignment

Congratulations! You have been selected for the **AI Research & Development Intern** role. The first stage of the selection process is a take-home assignment.

Choose **one** of the following two assignments:

1. Computer Vision - Efficient Expressway Image Filtering
2. Agentic AI - Natural Language to Database Query Agent

Submission window:

- 1 week from the date of Cynapse's assignment email.

No slide deck or presentation is required.

Option 1: Efficient Expressway Image Filtering

Objective

Develop a pipeline that filters a noisy road-image dataset and retains only images of Singapore expressways. The solution need not be fully automated but should minimize both human effort and computational cost. It should consider the trade-off between accuracy and computational complexity, with simple and fast solutions preferred over complex approaches that provide only marginal accuracy improvements.

You may use any machine-learning or computer-vision approach, including but not limited to:

- Conventional computer-vision techniques.
- Image classifiers or object detectors.
- Semantic segmentation or lane-detection models.
- CLIP-style models.
- Vision-Language Models such as Gemini, GPT, or Qwen-VL.
- A combination of fast and slower models.

Filtering Requirements

Retain only images depicting Singapore expressways, which should generally contain:

- Three to six traffic lanes per direction.
- Road shoulders, although they may not always be visible.

The following are optional but acceptable:

- Entry and exit ramps.
- ERP gantries and signage.

Reject images containing:

- Signal-controlled intersections or traffic lights.
- Yellow box-junction markings.
- Bus stops, bus bays, or dedicated bus lanes.
- Double yellow lines.
- Clearly identifiable urban, local, or non-expressway roads.

Dataset

[Dataset link](#)

Example expressway images:

- road (209) to road (215)
- road (232) to road (237)

Evaluation Criteria

Speed, Scalability, and Model Choice — 60%

- Write-up for model/pipeline choice and justification.
- Latency, throughput, compute, memory, and API cost.
- Quality of the speed–accuracy trade-off; better-balanced solutions receive higher scores.

Clarity of Solution and Code — 30%

- Concise, readable, and well-organized implementation; unnecessary complexity or over-engineering will be penalized.

Correctness — 10%

- 10% is awarded if $F1 \geq 0.80$; otherwise, 0%. Higher F1 scores receive no additional credit.

Option 2: Natural Language to Database Query Agent

Objective

Design and implement an agentic framework that translates natural-language requests into database queries and returns the matching CCTV frame records. The solution need not be fully agentic; deterministic tools or logic may be used where they improve robustness, reliability, or simplicity.

The solution should handle ambiguity, conversational follow-ups, date/time reasoning, camera-name variations, and appropriate guardrails.

The database may use SQL or NoSQL.

Dataset

Assume CCTV cameras capture one frame every 5 minutes throughout 2026, up to the current date.

Each record contains:

- Frame ID [int]
- Datetime [ISO 8601]
- Camera name [string Enum]
 - Pan Island Expressway (PIE)
 - Ayer Rajah Expressway (AYE)
 - East Coast Parkway (ECP)
 - Central Expressway (CTE)
 - Tampines Expressway (TPE)
 - Kallang-Paya Lebar Expressway (KPE)
 - Seletar Expressway (SLE)
 - Bukit Timah Expressway (BKE)
 - Kranji Expressway (KJE)
 - Marina Coastal Expressway (MCE)

You may generate or simulate the database records required for testing.

Functional Requirements

1. Natural-Language Querying

Translate natural-language requests into appropriate database queries.

Examples:

- "Show me frames from CTE today."
- "Show me frames from Tampines Expressway for the whole of August."
- "Show me frames from MCE on every Tuesday."
- "Show me frames from Kranji Highway from the 15th to 18th of last month."
- "Show me PIE frames between 8 AM and 10 AM yesterday."

2. Date Time Resolution

The system should support:

- Exact dates
- Relative dates such as today, yesterday, last week, this month
- Date ranges
- Time ranges
- Recurring conditions such as every Tuesday
- Combinations of camera and datetime constraints

3. Camera Name Resolution

The system should support:

- Full names - "Central Expressway"
- Acronyms - "CTE"
- Common variations - "Kranji Highway"
- Reasonable typos - "Tampines Expresway"

4. Conversational Follow-Ups

The agent should maintain relevant context across turns. Example:

User: "Show me frames from CTE."

User: "How about only those from this week?"

The second request should retain the CTE constraint while adding the new date constraint.

5. Guardrails

The system should safely handle requests outside its intended scope.

Examples include:

- Unrelated questions
- Requests for unsupported information
- Malicious or sensitive requests
- Prompt injection
- Attempts to execute arbitrary SQL/database commands
- Requests to modify or delete database records

Evaluation

Agent Design and Robustness — 40%

- Write-up on agent architecture, tool/model choices, and justification.
- Appropriate handling of context, ambiguity, guardrails, and query validation.
- Quality of the reliability–complexity trade-off; unnecessary agentic complexity or excessively long system prompts will be penalized.

Clarity of Solution and Code — 30%

- Concise, readable, and well-organized implementation; unnecessary abstractions, frameworks, or over-engineering will be penalized.

Correctness — 30%

- Common Queries — 20%: Correct handling of standard camera, date/time, range, recurring, and conversational queries.
- Edge Cases — 10%: Correct handling of typos, ambiguity, unusual date expressions, invalid requests, guardrails, and challenging follow-up queries.

Good luck!